Generating Music with Machine Learning

UE24CS352A Machine Learning Mini-Project | PES University | Sec-H, Team H-10 | Problem 22
Rohan Agarwal (PES1UG24AM921), Rolla N Harsha Vardhan (PES1UG24AM446)
Code: github.com/PES1UG24AM921/ml-music-generation (private; shared with faculty and TAs)

1. Problem Statement

Generate new piano music from MIDI data using machine learning. Following the Stanford CS229 (2018) report *Music Composition with Machine Learning* (Kang, Kim, Ringdahl), we compare four approaches on the same data: a Naive-Bayes-like model, a vanilla neural network (MLP), an LSTM and a GRU encoder-decoder. Models are compared by how well they predict held-out music, and we inspect the MIDI files they generate.

2. Dataset

MAESTRO v3.0.0 (MIDI only), a collection of classical piano performances. Files are sorted by path and the first 100 are used, giving 100 songs and 360,047 onset events. Preprocessing with *pretty_midi*: notes starting within 30 ms of each other form one onset event, and only the 88 piano keys (MIDI 21-108) are kept. The split is 80/10/10 by song, so test songs are never seen in training. For the MLP and LSTM this gives 290,706 / 24,441 / 41,700 training / validation / test windows. We use 100 songs, not the full set, so every model trains in reasonable time on a laptop CPU.

3. Approach

Naive-Bayes-like: learns P(right-hand note | left-hand chord). Pitches below 60 are the left hand, and the chord is the pitch-class set of the left-hand notes at an event (the reference used 3 consecutive notes). Generation samples a right-hand note for each chord of an unseen song. **MLP:** the previous 32 notes (the top note of each event) are embedded, flattened, passed through one 1024-unit ReLU layer with dropout 0.3 and mapped to the 88 keys. **LSTM:** one LSTM layer (256 hidden units, 64-dim embedding) over the previous 32 notes, then a softmax over 88 keys. **Encoder-decoder:** a 2-layer GRU encoder and decoder (256 hidden units) that reads 32 notes and predicts the next 8, trained with teacher forcing 0.5.

4. Implementation

Python, PyTorch and *pretty_midi*. Data code is in *src/data_prep.py*, models in *src/models.py*, training in *src/train.py*, sampling in *src/generate.py* and the Naive Bayes model in *src/naive_bayes.py*. Neural models use cross-entropy loss, Adam (lr 1e-3), batch size 128, gradient clipping at 1.0, 15 epochs, and keep the checkpoint with the best validation loss. Generation seeds a model with the first 32 notes of an unseen test song, then samples 200 notes with temperature 0.9. Durations and velocities are not modelled, so generated notes use a fixed time step. Exact commands to reproduce every result are in the repository README.

5. Results

Model	Train loss	Test loss	Test accuracy	Overfitting (best val loss epoch)
Naive Bayes	-	-	12.3% exact pitch (19.4% pitch class)	Train 12.8% vs test 12.3%: none
MLP	1.498	3.353	15.1% next-note	Strong (epoch 1)
LSTM	2.411	3.161	18.5% next-note	Moderate (about epoch 5)
Encoder-decoder	2.167	3.514	11.9% per token (8 notes)	Strong (epoch 1)

On the same test events, always guessing the most common note scores 5.4% and a flat random guess scores 1.7%, so Naive Bayes is clearly better than chance. The accuracies are not directly comparable: Naive Bayes counts a hit on any right-hand note of an event, the MLP and LSTM predict the top note of the next event, and the encoder-decoder is scored per token over 8 predicted notes. Losses are cross-entropy over 88 keys (chance is about 4.5).

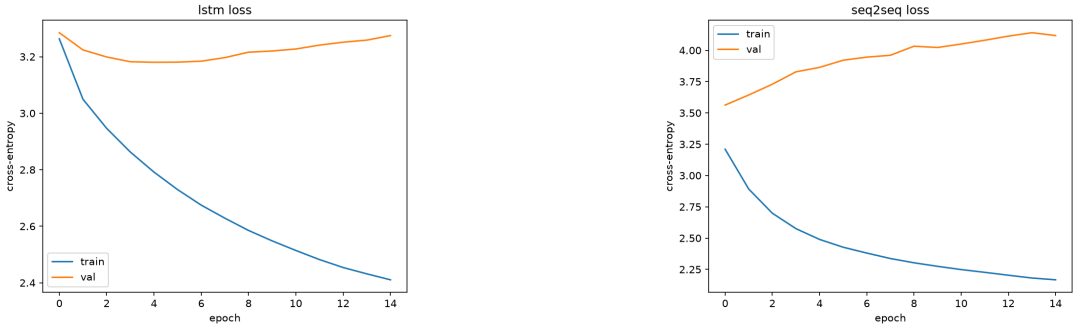


Figure 1: Training and validation loss for the LSTM (left) and encoder-decoder (right). Validation loss rises while training loss falls, which is overfitting.

Generated music. Every neural model is seeded with the same 32 notes from one test song, so the first 32 notes of each file match. Counting distinct pitches in the 200 generated notes: the MLP uses only 4, with long runs of a single repeated note; the LSTM uses 41 and the encoder-decoder 28. This agrees with the reference report, where the MLP also repeated

the same notes. These are statistics of the note data, not a listening survey.

6. Conclusions

The LSTM performed best (18.5% test accuracy, test loss 3.161) and overfit less than the MLP and encoder-decoder, and its generated music is the most varied. Every neural model overfits on 100 songs: validation loss bottoms out early while training loss keeps falling. The encoder-decoder was the weakest, as in the reference, where it produced repetitive output. Naive Bayes generalises modestly and only models harmony from the left hand. Limitations: only 100 songs, only the top note of each event is modelled by the neural models, durations and velocities are ignored, and next-note accuracy does not measure musical quality. Future work: more data and regularisation, a Transformer, note velocity, and a human listening survey.

7. Contributions

Rohan: environment and data setup, Naive Bayes and MLP runs and results, README, write-up. Rolla N Harsha Vardhan: LSTM and encoder-decoder training, results, loss plots and generated MIDI, presentation slides.